Contributors :
Name : Heramb Agashe , MIS : 612415006
Name : Anush Bhandari , MIS : 612415024
Name : Aryan Ballal , MIS : 612415021

Mini Banking System

A Database-Heavy Backend for Exploring ACID Transactions, Concurrency, Locking, Isolation Levels, and Financial Ledger Design

---

This document explains the design, goals, and architecture of the Mini Banking System project — a backend built not to maximize features, but to demonstrate correct, production-grade handling of money movement inside a relational database.

Aspect	Detail
Focus	Database correctness, not feature count
Backend	Spring Boot + Spring Data JPA / Hibernate
Database	PostgreSQL
Frontend	React
Core theme	ACID transactions applied to real money transfers

1. Project Overview

The Mini Banking System is a backend project that simulates real banking operations — deposits, withdrawals, and transfers — with a heavy emphasis on database engineering rather than UI or feature breadth. The intent is to use a realistic, high-stakes domain (money) to study how a backend should correctly handle transactions, concurrency, isolation, locking, indexing, and auditability.

A simple CRUD banking app would hide the interesting problems. This project instead makes those problems the point: how do you guarantee that a transfer never leaves one account debited without the other being credited, even under concurrent load, partial failures, or retried requests?

1.1 Goals

- Maintain correct account balances at all times
- Support atomic money transfers between accounts
- Prevent double spending and lost updates under concurrency
- Demonstrate all four SQL transaction isolation levels in practice
- Compare optimistic locking against pessimistic locking
- Deliberately produce and then resolve a classic deadlock
- Design indexes driven by real query patterns, not guesswork
- Keep a complete, auditable transaction and event history
- Guarantee idempotency so retried requests can never double-charge a customer
- Maintain a consistent, reconcilable financial ledger

2. Core Features

2.1 Customer & Account Management

- Customer registration and profile management
- Multiple accounts per customer (Savings / Current)
- Account activation, deactivation, and freezing
- Balance inquiry, deposits, and withdrawals

2.2 Money Transfers

A transfer between two accounts is the central operation of the system and must be atomic: both the debit and the credit succeed together, or neither happens. The database must never be left in a state where only one leg of a transfer has completed.

```
Account A: Rs 10,000        Account B: Rs 5,000
Transfer Rs 2,000  A -> B

BEGIN TRANSACTION
      |
Lock Account A
      |
Lock Account B
      |
Debit A by Rs 2,000
      |
```

```
Credit B by Rs 2,000
      |
Create transaction record
      |
    COMMIT

  (any failure before commit -> ROLLBACK)
```

3. ACID Transactions

Every transfer is designed to demonstrate the four ACID properties in a concrete, observable way:

- **Atomicity** — the debit and credit are one indivisible unit of work; they succeed or fail together.
- **Consistency** — database constraints and business rules (e.g. balance can't go negative) always hold.
- **Isolation** — concurrent transfers do not see or corrupt each other's intermediate state.
- **Durability** — once a transfer commits, it survives crashes and restarts.

3.1 Transaction Isolation Levels

The project experiments with all four standard SQL isolation levels and documents which classic anomalies each one allows or prevents:

Isolation Level	Anomalies It Can Still Allow
READ UNCOMMITTED	Dirty reads, non-repeatable reads, phantom reads
READ COMMITTED	Non-repeatable reads, phantom reads
REPEATABLE READ	Phantom reads (implementation-dependent)
SERIALIZABLE	None — full serial-equivalent execution

Lost updates — where two concurrent transactions both read the same balance and one overwrite silently clobbers the other's change — are a special focus, since this is the exact failure mode a banking system cannot tolerate.

3.2 Concurrency Control

The system explores and compares two complementary strategies for protecting balances under concurrent access:

- **Pessimistic locking** — using `SELECT ... FOR UPDATE` to acquire a row-level lock before reading a balance, so no other transaction can read or modify it until commit.
- **Optimistic locking** — using a `version` column on the account row; a concurrent write is detected and rejected at commit time if the version has changed, avoiding a held lock at the cost of retries under contention.

Example race condition without proper control:

```
Initial balance = Rs 10,000

T1 -> Withdraw Rs 8,000
T2 -> Withdraw Rs 7,000

Without concurrency control, both transactions can read the
```

```
same starting balance and both succeed -- overdrawing the account.
```

4. Deadlocks

Locking two accounts in an inconsistent order creates a classic circular-wait deadlock. The project deliberately reproduces this failure so it can be understood, then eliminates it with a simple rule.

```
T1: Lock Account A  ->  Wait for Account B
T2: Lock Account B  ->  Wait for Account A
        (circular wait -> deadlock)
```

4.1 Prevention Strategy

- Always acquire account locks in a consistent order (ascending account ID)
- Keep transactions as short as possible
- Detect failed transactions and retry where it is safe to do so
- Avoid acquiring locks that are not strictly necessary

```
Transfer A -> B :  Lock min(A, B)  then  Lock max(A, B)
Transfer B -> A :  Lock min(A, B)  then  Lock max(A, B)

Both directions now request locks in the same order,
which removes the circular wait entirely.
```

5. Database Schema

5.1 customers

Column	Description
id	Primary key
name	Customer name
email	Unique email
phone	Phone number
created_at	Registration timestamp

5.2 accounts

Column	Description
id	Primary key
customer_id	Account owner
account_number	Unique account number
account_type	SAVINGS / CURRENT
balance	Current balance
status	ACTIVE / FROZEN / CLOSED
version	Optimistic locking version
created_at	Account creation timestamp

5.3 transactions

Column	Description
id	Primary key
from_account_id	Source account
to_account_id	Destination account
amount	Transfer amount
transaction_type	DEPOSIT / WITHDRAWAL / TRANSFER
status	PENDING / SUCCESS / FAILED / REVERSED
idempotency_key	Prevents duplicate requests
created_at	Transaction timestamp

5.4 beneficiaries

Column	Description

id	Primary key
customer_id	Beneficiary owner
account_id	Beneficiary account
nickname	User-defined name
created_at	Creation timestamp

5.5 audit_logs

Column	Description
id	Primary key
transaction_id	Related transaction
event	Event type
metadata	Additional information
created_at	Event timestamp

6. Entity Relationship Diagram

The diagram below shows how the five core tables relate to one another. A customer can own many accounts and many beneficiaries; an account can appear on either side of many transactions (as source or destination); and every transaction can generate one or more audit log entries.

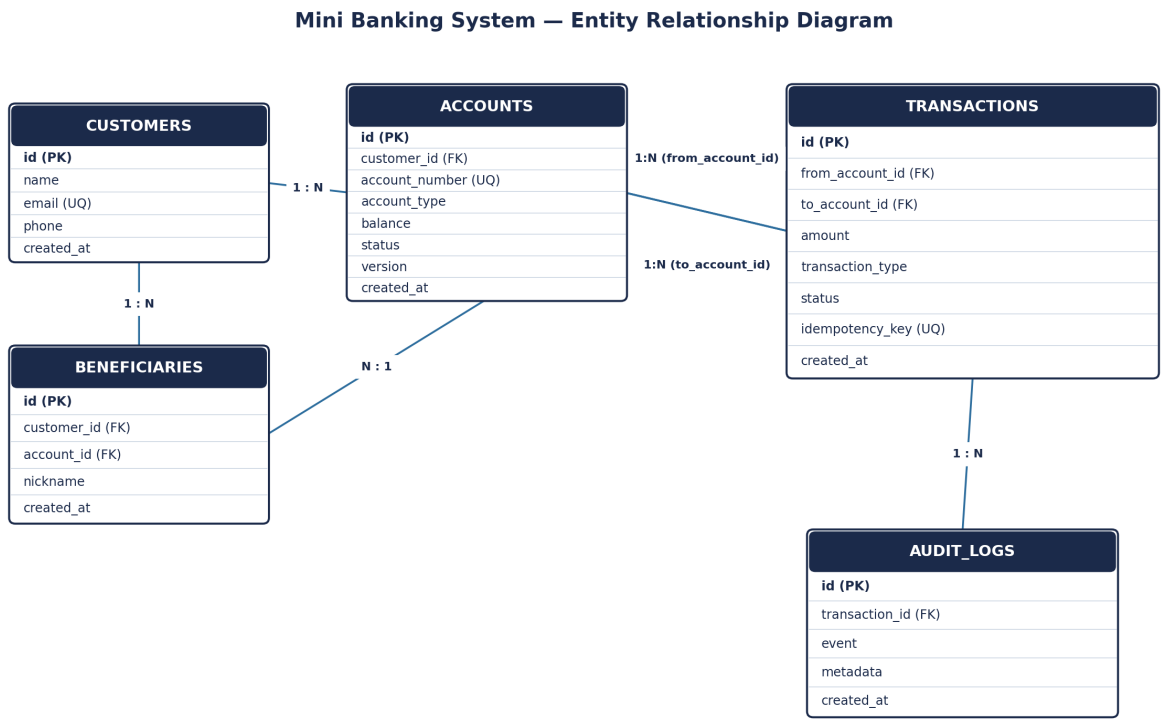


Figure 1 — ER diagram of customers, accounts, transactions, beneficiaries, and audit_logs.

6.1 Relationship Summary

- CUSTOMER (1) — (N) ACCOUNT
- CUSTOMER (1) — (N) BENEFICIARY
- ACCOUNT (1) — (N) TRANSACTION, via `from_account_id`
- ACCOUNT (1) — (N) TRANSACTION, via `to_account_id`
- TRANSACTION (1) — (N) AUDIT_LOG

7. Double-Entry Ledger

For a more realistic implementation, the system moves beyond treating `balance` as the single source of truth. Instead, every monetary movement is represented by matching debit and credit ledger entries, and the account balance becomes a derived value that must always reconcile against the ledger.

```
Transfer Rs 2,000 from A to B

Account A   DEBIT   Rs 2,000
Account B   CREDIT  Rs 2,000
```

This gives the system a reliable, append-only trail for reconciliation and auditing, independent of whatever the current balance column says.

8. Idempotency

Network failures can cause a client to retry a request whose response never arrived — even though the transfer actually succeeded on the server. Without protection, this can charge a customer twice.

```
Client -> Transfer Rs 2,000
Server -> Transfer succeeds
Response is lost
Client -> Retries the same request
```

Every transfer request carries an `idempotency_key`, and the database enforces uniqueness on that column. If the same key arrives again, the system returns the original transaction's result instead of creating a second transfer.

9. Indexing Strategy

Indexes are chosen from actual query patterns rather than added speculatively. Candidate indexes for this schema include:

```
accounts(account_number)
accounts(customer_id)
transactions(from_account_id, created_at)
transactions(to_account_id, created_at)
transactions(status)
transactions(idempotency_key)
```

Query plans are inspected with `EXPLAIN` to compare indexed lookups against sequential scans, and to evaluate composite indexes, selectivity, and pagination performance on large transaction histories.

9.1 Transaction History Queries

The API supports retrieving:

- Recent transactions and transactions between two dates
- Deposits, withdrawals, and transfers individually
- All transactions involving a specific account
- Monthly transaction summaries

```
GET /accounts/{id}/transactions?page=0&size=20
```

10. Security

- Authentication via Spring Security + JWT
- Password hashing for stored credentials
- Role-based authorization — CUSTOMER and ADMIN roles
- Sensitive operations recorded in the audit log

11. Technology Stack

Component	Technology
Frontend	React
Backend	Spring Boot
ORM	Spring Data JPA / Hibernate
Database	PostgreSQL
Authentication	Spring Security + JWT
Testing	JUnit
Integration Testing	Testcontainers
Build	Maven

PostgreSQL is chosen specifically for its strong transaction and concurrency-control features, which make it well suited to experimenting with isolation levels and locking behavior.

12. Backend Architecture

```
React
  |
  v
REST API
  |
  v
Spring Security / JWT
  |
  v
Controllers
  |
  v
Services
  |
  v
Transaction Management
  |
  v
Repositories / Hibernate
  |
  v
PostgreSQL
```

The transaction boundary is placed around whole business operations — such as a fund transfer — rather than around each individual database call, so the operation is atomic as a unit.

13. Example Transfer Flow

```
POST /transfers
      |
      v
Validate JWT
      |
      v
Validate request
      |
      v
Check idempotency key
      |
      v
BEGIN TRANSACTION
      |
      v
Lock source + destination accounts
      |
      v
Validate account status
      |
      v
Validate sufficient balance
      |
      v
Debit source account
      |
```

```
      v
Credit destination account
      |
      v
Create transaction record
      |
      v
Create audit log
      |
      v
    COMMIT
      |
      v
  Return result

  (any failure before commit -> ROLLBACK)
```

14. Advanced / Future Features

- Scheduled transfers and transaction reversal
- Daily transfer limits and account freezing rules
- Fraud detection rules and transaction reconciliation
- Database backup / recovery and transaction retry mechanisms
- Read replicas and caching
- Rate limiting and event-driven transaction notifications

15. Learning Objectives

After completing the project, the developer should be able to explain:

- How ACID transactions work in a real application
- Why money transfers require atomic transactions
- How transaction isolation affects concurrent operations
- The difference between optimistic and pessimistic locking
- How row-level locks work
- How deadlocks occur, and why
- How consistent lock ordering prevents deadlocks
- How database indexes improve query performance
- Why idempotency is essential in financial APIs
- Why audit logs matter for security and compliance
- How double-entry bookkeeping improves financial correctness
- How Spring's transaction management interacts with PostgreSQL

16. Project Philosophy

The primary goal of this project is **database correctness rather than feature count**. A smaller system with correctly implemented transactions, locking, isolation, constraints, indexing, idempotency, and auditing is more valuable here than a large banking UI built on simple, unguarded CRUD operations.

16.1 Future Database Experiments

The project is designed to support head-to-head experiments such as:

```
No locking   vs   Optimistic locking   vs   Pessimistic locking

READ COMMITTED   vs   REPEATABLE READ   vs   SERIALIZABLE
```

Each experiment is expected to document the observed behavior, performance characteristics, and consistency guarantees of the approach under test.